

VisionInspect AI

Frontend Development – Milestone 1 Report

Himabindhu Ravuri

Infosys Springboard Internship

Table of Contents

1. Project Description
2. Tech Stack & Environment Setup
3. Requirement Analysis
4. Frontend Development Process
5. Proposed System / Architecture
6. Results (Screenshots)
7. Current Limitations & Next Steps
8. Conclusion

1. Project Description

VisionInspect AI is a system that checks product images for defects using computer vision, and tells us whether a product passes or fails quality check. It's being built by our 5-member team as part of the Infosys Springboard internship.

My role is Frontend Development and API Integration. I build the pages that Quality Engineers and Factory Supervisors will actually use — uploading images, viewing results, checking quality trends — and connect these pages to the backend once it's ready.

2. Tech Stack & Environment Setup

I built this using VS Code, along with:

- React (with Vite) – for building the pages
- Tailwind CSS – for styling
- React Router – for switching between pages
- React Context API – for remembering who's logged in
- JWT Authentication – to connect securely to the backend's login

3. Requirement Analysis

Before I started building, I checked the Milestone 1 (Week 1-2) requirements from our project document. For frontend, this meant:

- Set up the frontend project
- Add login with role selection (Quality Engineer / Factory Supervisor)
- Build image upload (single and batch)
- Build a working inspection dashboard

4. Frontend Development Process

4.1 Project Initialization

I set up the project using Vite's React template and added Tailwind CSS for styling. ESLint helps catch small mistakes in my code as I go.

4.2 Authentication Module

I built a login page where you enter a username and password, and pick a role. At first, I used a fake/placeholder login just to test things. Once my backend teammate shared her login API details, I connected it for real — it uses JWT (a secure token) generated by her backend using PyJWT.

4.3 Image Upload / Inspection Module

I built an Inspection page where you can upload one or more images and see a preview before running the check. Clicking "Run Inspection" sends the images off to be analyzed. Since the AI model isn't ready yet, this currently uses fake results — but I used the same severity scoring formula from our project spec (Size 30% + Location 25% + Defect Type 25% + Confidence 20%), so the numbers behave the way they're supposed to.

4.4 Dashboard, History & Analytics Modules

The Dashboard page shows quick stats — total inspected, passed, failed, critical. The History page shows every past inspection in a table. Analytics is just a placeholder for now — real charts will come later once we have enough real inspection data.

4.5 Navigation

I built a Sidebar and a shared Layout so every page has the same navigation menu, instead of repeating that code on each page.

5. Proposed System / Architecture

Here's how everything connects:

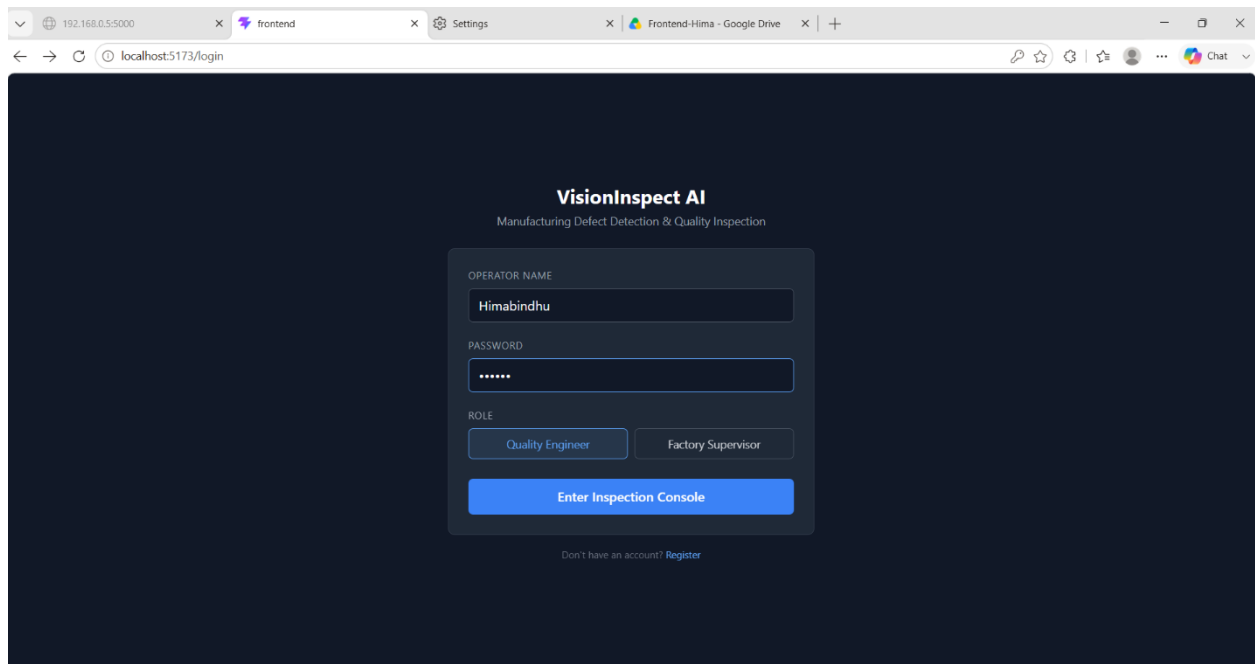
- Login → username/password sent to backend → real JWT token comes back and gets stored
- Upload image on Inspection page → image sent for checking (currently fake/mock)
- Result comes back (defect type, severity, pass/fail) → shown to user and saved to history
- Dashboard and History pages just read this saved data and display it

All the backend-related code lives in one file (api.js). This means once the real AI model and backend are ready, I just update that file — the actual pages don't need to change.

6. Results (Screenshots)

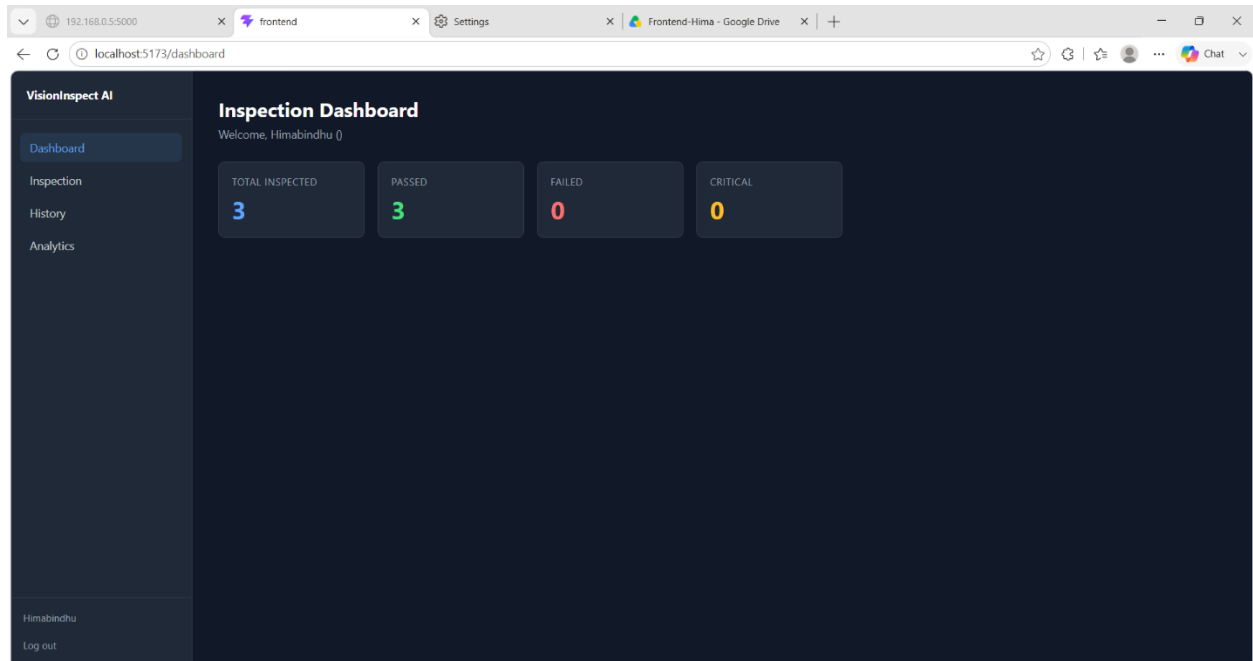
6.1 Login Page

Connected to the real backend login API, using JWT authentication.



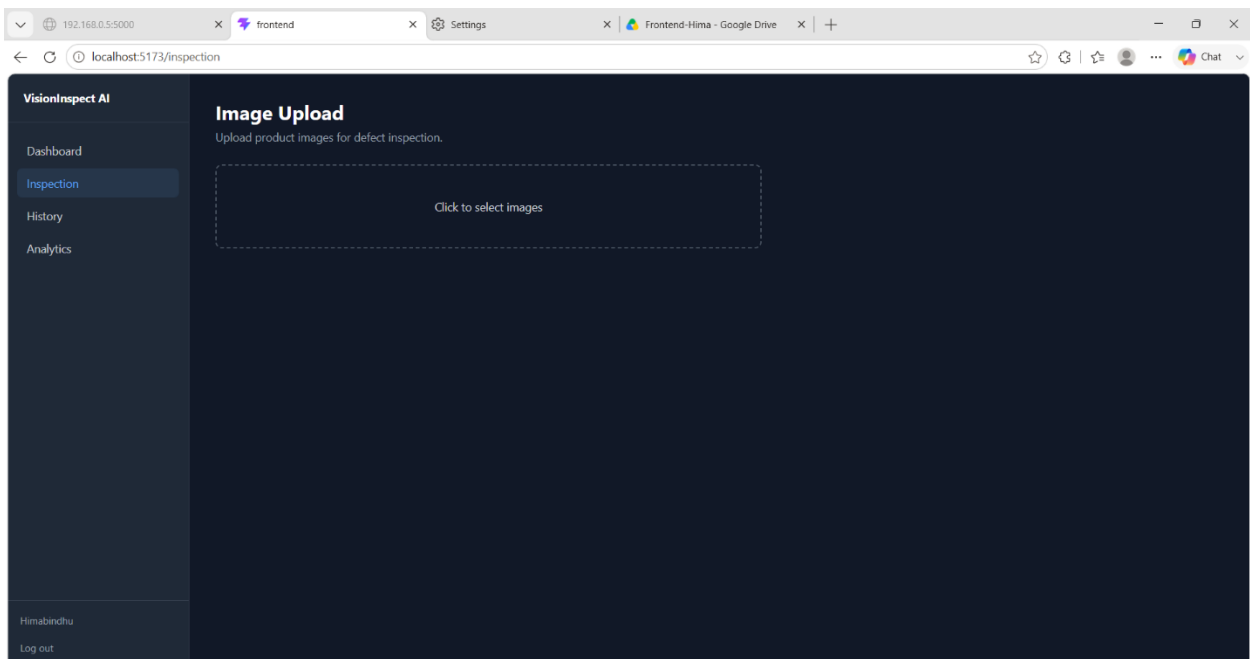
6.2 Dashboard Page

Shows quick stats of all inspections done so far.



6.3 Inspection Page

Shows uploaded images along with mock inspection results (defect type, severity, confidence, result).



6.4 History Page

Shows a full list of all past inspections, with file name, defect type, severity score, and pass/fail result for each one.

VisionInspect AI

Dashboard

Inspection

History

Analytics

Himabindhu
Log out

Inspection History

Full record of past inspections.

FILE	DEFECT	SEVERITY	RESULT
002.png	Surface Scratch	Medium - 46	PASS
001.png	Dent	Medium - 58	PASS
000.png	Surface Scratch	Medium - 58	PASS

6.5 Analytics Page

A placeholder page for now — will show defect trends and quality reports once there's enough real inspection data to work with.

VisionInspect AI

Dashboard

Inspection

History

Analytics

Himabindhu
Log out

Analytics

Defect trends and production quality reports will appear here.

Coming soon — charts will be added once we have enough inspection data and finalize what metrics to track.

7. Current Limitations & Next Steps

- Inspection results are still mock/fake data, since the AI model hasn't been trained or connected yet.
- Role-based access isn't built yet — both roles currently see the same pages. Waiting on the team to decide what each role should or shouldn't access.
- Once the AI model's API is ready, I'll swap the mock logic in `api.js` for a real API call — no other changes needed.
- Analytics page will get real charts once we have enough real inspection data.

8. Conclusion

I've completed everything Milestone 1 asked for on the frontend side — project setup, login with role selection, image upload, and a working dashboard. I built it so that connecting the real backend and AI model later needs only small changes, not a rebuild. Real JWT login is already working with the backend, and the app is ready to plug in the real AI model as soon as it's available.